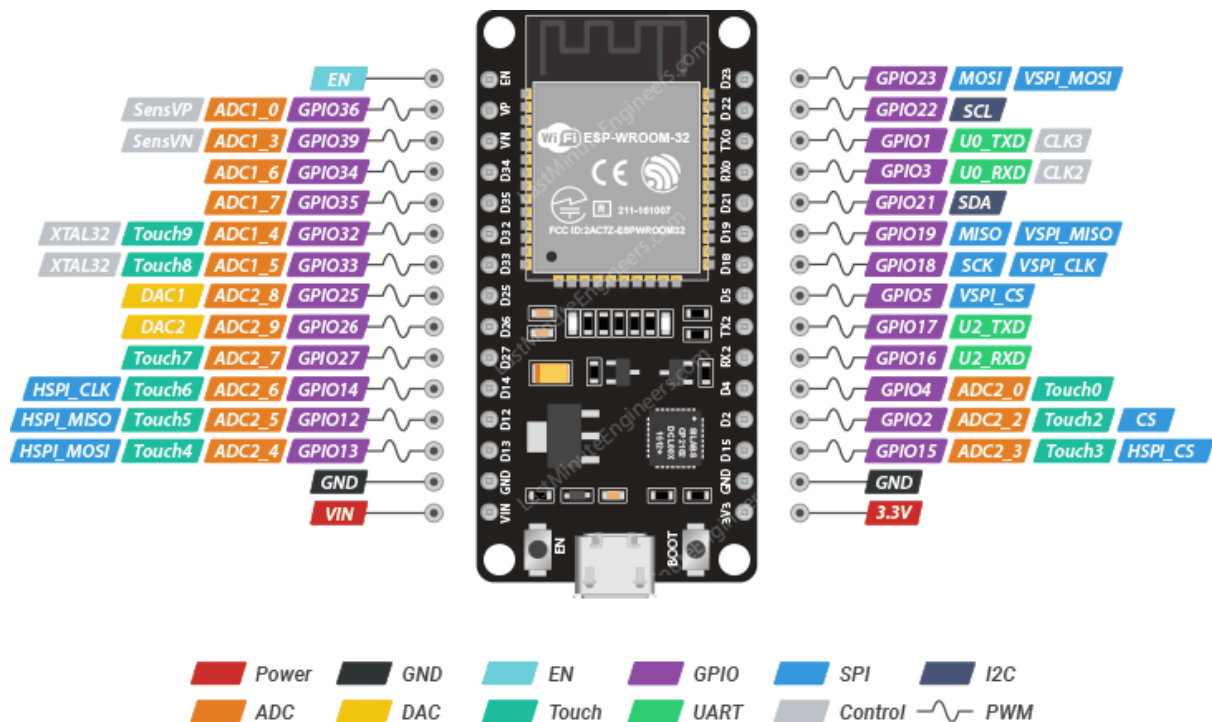


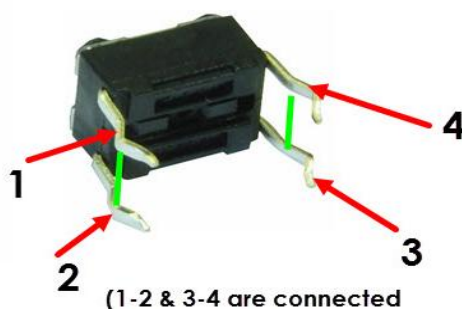
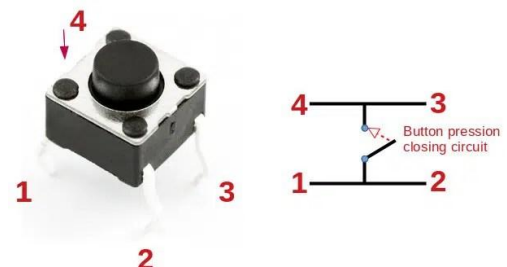
AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

<https://lastminuteengineers.com/esp32-pinout-reference/>

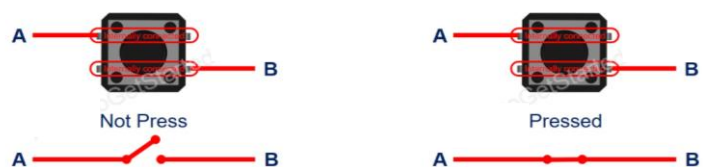


1) To interface a push button with an ESP32 and display the button status (PRESSED or RELEASED) on the Serial Monitor.



How It Works

- When the button is NOT pressed, pin A is NOT connected to pin B
- When the button is pressed, pin A is connected to pin B



However, these pins are internally connected in pairs. Therefore, we only need to use two of the four pins, which are NOT internally connected.

There are four ways (actually two ways because of symmetry) to connect to button (see image)

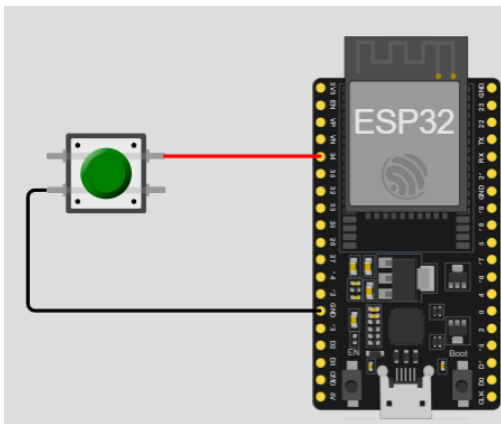


We can use only two pins of a button, why does it have four pins?

⇒ To make it stand firmly in PCB (board) to resist the pressing force.

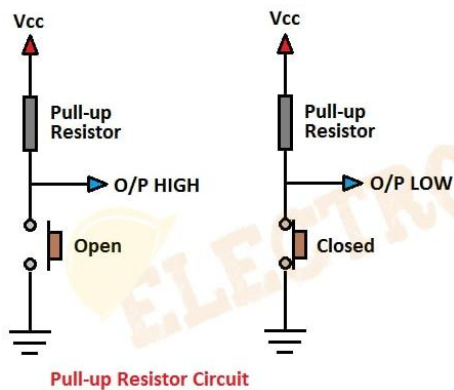
AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture



```
1  const int BUTTON_PIN = 34;
2
3  void setup()
4  {
5      Serial.begin(115200);
6
7      // Configure the button pin with internal pull-up resistor
8      pinMode(BUTTON_PIN, INPUT_PULLUP);
9
10     Serial.println("ESP32 Push Button Interface");
11 }
12
13 void loop()
14 {
15     int buttonState = digitalRead(BUTTON_PIN);
16
17     if(buttonState == LOW)
18     {
19         Serial.println("BUTTON PRESSED");
20     }
21     else
22     {
23         Serial.println("BUTTON RELEASED");
24     }
25
26     delay(200);
27 }
```

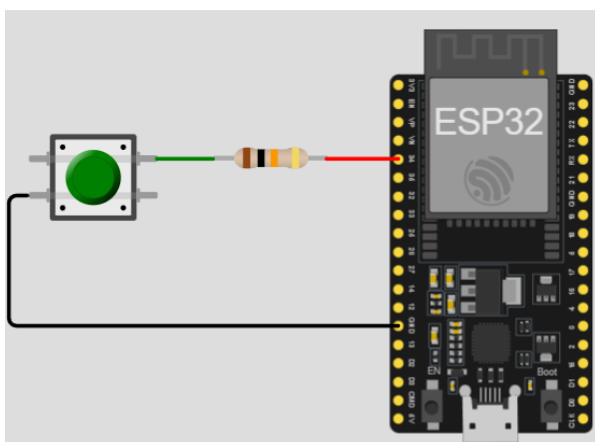
Pull-up Resistors



If we connect the push button switch directly to Esp32 to get digital input, It means switch one pin is connected to Ground or 3.3v Vcc and another pin connected to Esp32 digital pin. In this case, the Esp32 is read **unstable input** from the push button.

So, we need to connect a “pull-up” or “pull-down” resistors circuit to **stabilizes the input**, when using the switch.

If the push button one pin is connected to the Vcc through a resistor and the other pin is connected to the ground, this circuit known as the **pull-up resistor circuit**. In this case, the push button output is **High (1)** when the button is **open**, and the output of the push button is **Low (0)** when the **button is pressed**.

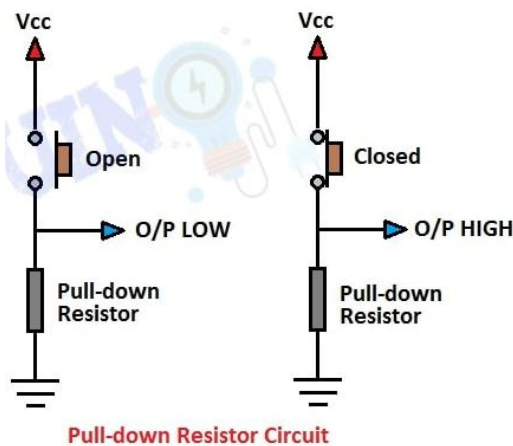


```
1  const int BUTTON_PIN = 34;
2
3  void setup()
4  {
5      Serial.begin(115200);
6
7      // Configure the button pin with External pull-up resistor
8      pinMode(BUTTON_PIN, INPUT);
9
10     Serial.println("ESP32 Push Button Interface");
11 }
12
13 void loop()
14 {
15     int buttonState = digitalRead(BUTTON_PIN);
16
17     if(buttonState == LOW)
18     {
19         Serial.println("BUTTON PRESSED");
20     }
21     else
22     {
23         Serial.println("BUTTON RELEASED");
24     }
25
26     delay(20);
27 }
```

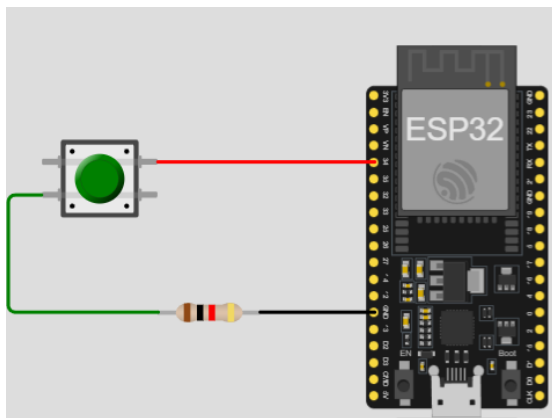
AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

Pull-Down Resistors

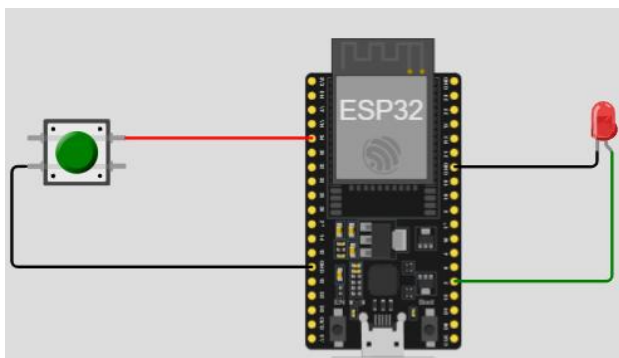


If the push button one pin is connected to the **ground through a resistor** and the other pin is connected to the Vcc, this circuit known as the **pull-down resistor circuit**. In this case, the push button output is **Low(0)** when the **button is open**, and the output of the push button is **High(1)** when the **button is pressed**.



```
1  const int BUTTON_PIN = 34;
2
3  void setup()
4  {
5      Serial.begin(115200);
6
7      // Configure the button pin with External pull-down resistor
8      pinMode(BUTTON_PIN, INPUT);
9
10     Serial.println("ESP32 Push Button Interface");
11 }
12
13 void loop()
14 {
15     int buttonState = digitalRead(BUTTON_PIN);
16
17     if(buttonState == LOW)
18     {
19         Serial.println("BUTTON PRESSED");
20     }
21     else
22     {
23         Serial.println("BUTTON RELEASED");
24     }
25
26     delay(20);
27 }
```

- 2) To interface a push button and an LED with an ESP32. When the push button is pressed, the LED should turn ON. When the push button is released, the LED should turn OFF. The button status should also be displayed on the Serial Monitor.



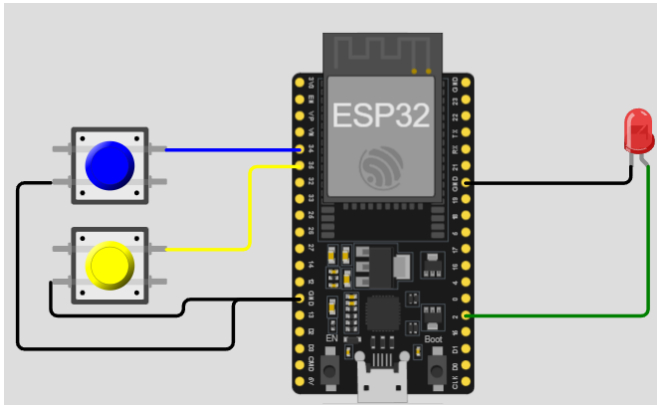
```
1  const int BUTTON_PIN = 34;
2  const int LED_PIN = 2;
3
4  void setup()
5  {
6      Serial.begin(115200);
7
8      // Configure button as input with internal pull-up
9      pinMode(BUTTON_PIN, INPUT_PULLUP);
10
11     // Configure LED as output
12     pinMode(LED_PIN, OUTPUT);
13
14     digitalWrite(LED_PIN, LOW);
15
16     Serial.println("ESP32 Push Button LED Control");
17 }
18
19 void loop()
20 {
21     int buttonState = digitalRead(BUTTON_PIN);
22
23     if(buttonState == LOW)
24     {
25         digitalWrite(LED_PIN, HIGH);
26         Serial.println("BUTTON PRESSED - LED ON");
27     }
28     else
29     {
30         digitalWrite(LED_PIN, LOW);
31         Serial.println("BUTTON RELEASED - LED OFF");
32     }
33
34     delay(200);
35 }
36
37 }
```

AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

3) To control the brightness of an LED using two push buttons and the PWM feature of the ESP32.

- Push Button 1: Increases the LED brightness step-by-step.
- Push Button 2: Decreases the LED brightness step-by-step.
- The current brightness value is displayed on the Serial Monitor.



```
1  const int LED_PIN = 2;
2  const int INC_BUTTON = 32;
3  const int DEC_BUTTON = 33;
4
5  int brightness = 0;
6  const int step = 25;
7
8  // Variables for button edge detection
9  bool lastIncState = HIGH;
10 bool lastDecState = HIGH;
11
12 void setup()
13 {
14     Serial.begin(115200);
15
16     pinMode(LED_PIN, OUTPUT);
17
18     pinMode(INC_BUTTON, INPUT_PULLUP);
19     pinMode(DEC_BUTTON, INPUT_PULLUP);
20
21     analogWrite(LED_PIN, brightness);
22
23     Serial.println("PWM Brightness Control");
24     Serial.print("Brightness = ");
25     Serial.println(brightness);
26 }
27
28 void loop()
29 {
30     bool incState = digitalRead(INC_BUTTON);
31     bool decState = digitalRead(DEC_BUTTON);
32
33     // Increase brightness (only once per press)
34     if(lastIncState == HIGH && incState == LOW)
35     {
36         brightness += step;
37
38         if(brightness > 255)
39             brightness = 255;
40
41         analogWrite(LED_PIN, brightness);
42
43         Serial.print("Increase Button Pressed -> Brightness = ");
44         Serial.println(brightness);
45
46         delay(50); // Debounce
47     }
48
49     // Decrease brightness (only once per press)
50     if(lastDecState == HIGH && decState == LOW)
51     {
52         brightness -= step;
53
54         if(brightness < 0)
55             brightness = 0;
56
57         analogWrite(LED_PIN, brightness);
58
59         Serial.print("Decrease Button Pressed -> Brightness = ");
60         Serial.println(brightness);
61
62         delay(50); // Debounce
63     }
64
65     lastIncState = incState;
66     lastDecState = decState;
67 }
```

AI Based Smart System Development (Real Time Hardware Implementation)

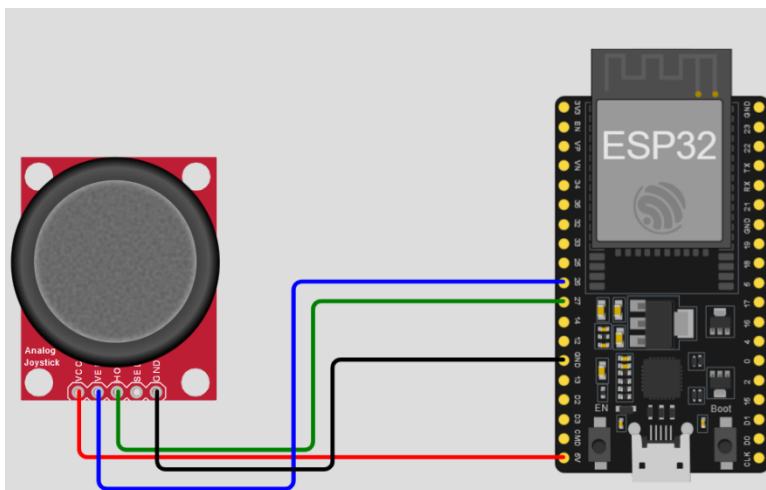
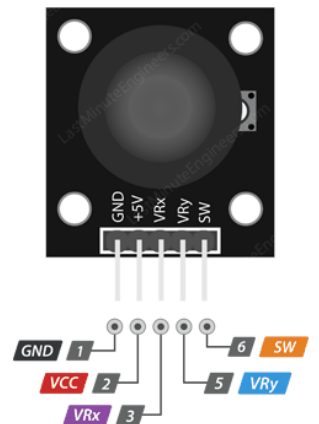
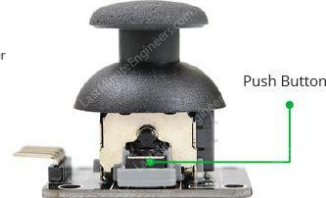
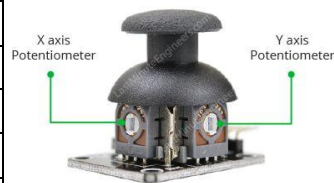
Week – 2 Lecture

4) To interface a two-axis joystick module with an ESP32 and display the joystick movement direction (Left, Right, Up, Down, and Center) on the Serial Monitor.

The joystick has two potentiometers:

- VRx controls the X-axis (Left and Right).
- VRy controls the Y-axis (Up and Down).

Joystick Position	ADC Value
Left	0–1000
Center	Around 2048
Right	3000–4095
Up	3000–4095
Down	0–1000



```

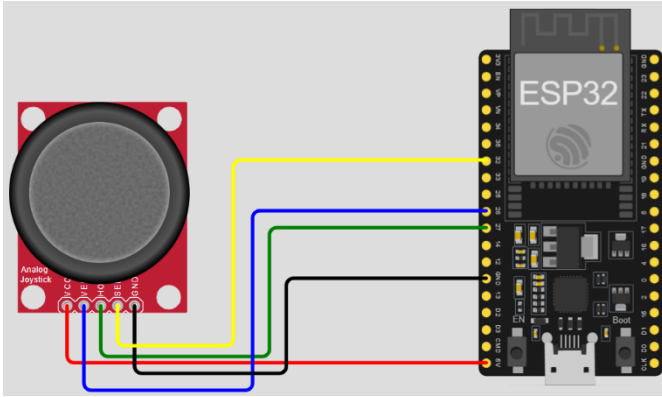
1  const int X_PIN = 27;
2  const int Y_PIN = 26;
3
4  void setup()
5  {
6      Serial.begin(115200);
7
8      Serial.println("ESP32 Joystick Direction Detection");
9  }
10
11 void loop()
12 {
13     int xValue = analogRead(X_PIN);
14     int yValue = analogRead(Y_PIN);
15
16     Serial.print("X = ");
17     Serial.print(xValue);
18
19     Serial.print(" Y = ");
20     Serial.print(yValue);
21
22     Serial.print(" Direction : ");
23
24     // Left
25     if (xValue < 1000)
26     {
27         Serial.println("LEFT");
28     }
29
30     // Right
31     else if (xValue > 3000)
32     {
33         Serial.println("RIGHT");
34     }
35
36     // Up
37     else if (yValue > 3000)
38     {
39         Serial.println("UP");
40     }
41
42     // Down
43     else if (yValue < 1000)
44     {
45         Serial.println("DOWN");
46     }
47
48     // Center
49     else
50     {
51         Serial.println("CENTER");
52     }
53
54     delay(500);
55 }

```

AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

- 5) To interface a two-axis joystick module with an ESP32 and display the joystick movement direction (Left, Right, Up, Down, and Center) on the Serial Monitor. When the joystick button is pressed, the Serial Monitor displays "BUTTON PRESSED".



```
1  const int X_PIN = 27;
2  const int Y_PIN = 26;
3  const int SW_PIN = 32;
4
5  void setup()
6  {
7      Serial.begin(115200);
8
9      pinMode(SW_PIN, INPUT_PULLUP);
10
11     Serial.println("ESP32 Joystick Direction Detection");
12 }
13
14 void loop()
15 {
16     int xValue = analogRead(X_PIN);
17     int yValue = analogRead(Y_PIN);
18
19     // Read joystick button
20     int buttonState = digitalRead(SW_PIN);
21
22     Serial.print("X = ");
23     Serial.print(xValue);
24
25     Serial.print("\tY = ");
26     Serial.print(yValue);
27
28     Serial.print("\tDirection : ");
29
30     // Determine joystick direction
31     if (xValue < 1000)
32     {
33         Serial.print("LEFT");
34     }
35     else if (xValue > 3000)
36     {
37         Serial.print("RIGHT");
38     }
39     else if (yValue > 3000)
40     {
41         Serial.print("UP");
42     }
43     else if (yValue < 1000)
44     {
45         Serial.print("DOWN");
46     }
47     else
48     {
49         Serial.print("CENTER");
50     }
51
52     // Display button status
53     if (buttonState == LOW)
54     {
55         Serial.print("\tBUTTON PRESSED");
56     }
57     else
58     {
59         Serial.print("\tBUTTON RELEASED");
60     }
61
62     Serial.println();
63
64     delay(200);
65 }
```

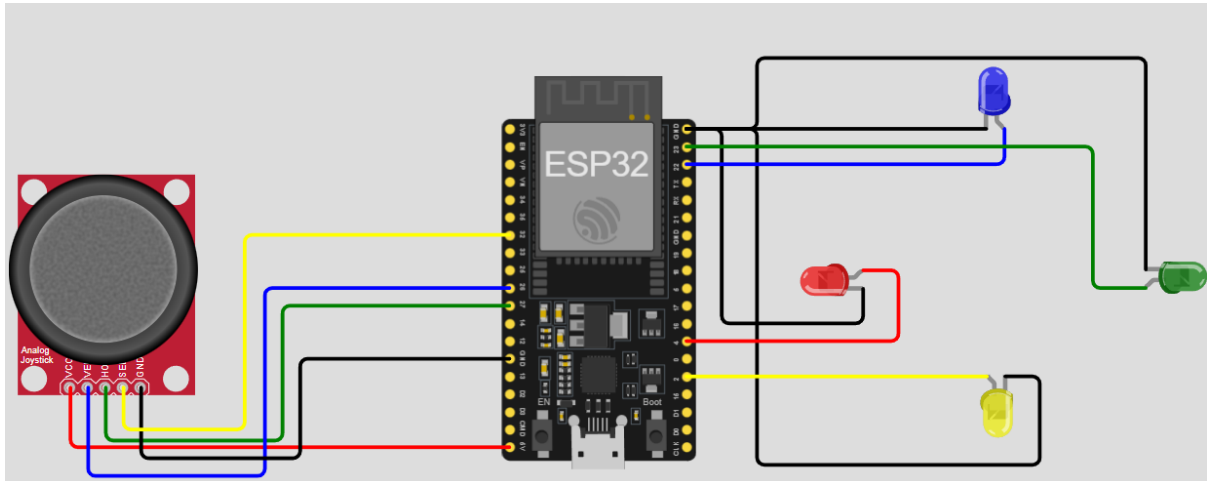
AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

6) To interface a joystick module with an ESP32 and control four different LEDs based on the joystick direction. The detected direction should also be displayed on the Serial Monitor.

The joystick outputs analog values on the X-axis and Y-axis:

- **Move Left** → Red LED ON
- **Move Right** → Green LED ON
- **Move Up** → Blue LED ON
- **Move Down** → Yellow LED ON
- **Center Position** → All LEDs OFF

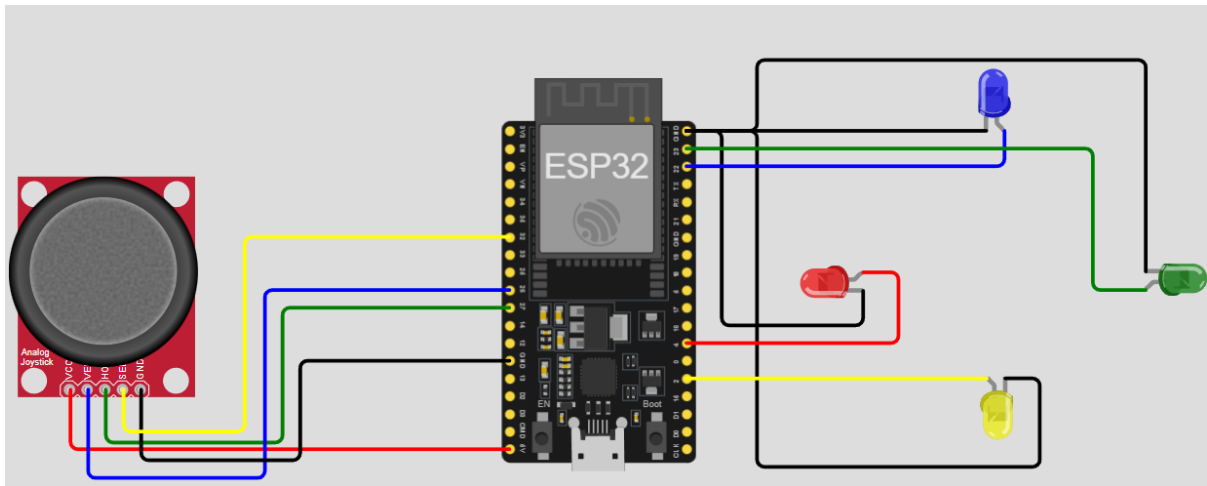


```
1  const int X_PIN = 27;
2  const int Y_PIN = 26;
3
4  const int RED_LED = 4;
5  const int GREEN_LED = 23;
6  const int BLUE_LED = 22;
7  const int YELLOW_LED = 2;
8
9  // Function to turn OFF all LEDs
10 void allLEDsOff()
11 {
12     digitalWrite(RED_LED, LOW);
13     digitalWrite(GREEN_LED, LOW);
14     digitalWrite(BLUE_LED, LOW);
15     digitalWrite(YELLOW_LED, LOW);
16 }
17
18 void setup()
19 {
20     Serial.begin(115200);
21
22     pinMode(RED_LED, OUTPUT);
23     pinMode(GREEN_LED, OUTPUT);
24     pinMode(BLUE_LED, OUTPUT);
25     pinMode(YELLOW_LED, OUTPUT);
26
27     allLEDsOff();
28
29     Serial.println("Joystick Controlled LED System");
30 }
31
32 void loop()
33 {
34     int xValue = analogRead(X_PIN);
35     int yValue = analogRead(Y_PIN);
36
37     allLEDsOff();
38
39     Serial.print("X = ");
40     Serial.print(xValue);
41
42     Serial.print(" Y = ");
43     Serial.print(yValue);
44
45     Serial.print(" Direction : ");
46
47     // LEFT
48     if(xValue < 1000)
49     {
50         digitalWrite(RED_LED, HIGH);
51         Serial.println("LEFT");
52     }
53
54     // RIGHT
55     else if(xValue > 3000)
56     {
57         digitalWrite(GREEN_LED, HIGH);
58         Serial.println("RIGHT");
59     }
60
61     // UP
62     else if(yValue > 3000)
63     {
64         digitalWrite(BLUE_LED, HIGH);
65         Serial.println("UP");
66     }
67
68     // DOWN
69     else if(yValue < 1000)
70     {
71         digitalWrite(YELLOW_LED, HIGH);
72         Serial.println("DOWN");
73     }
74
75     // CENTER
76     else
77     {
78         Serial.println("CENTER");
79     }
80
81     delay(200);
82 }
```

AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

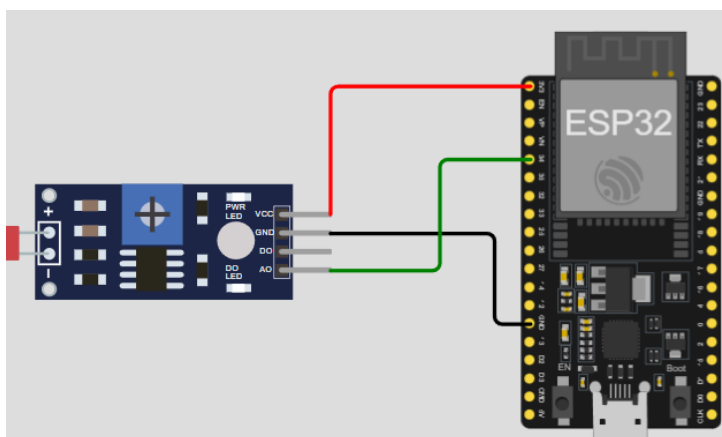
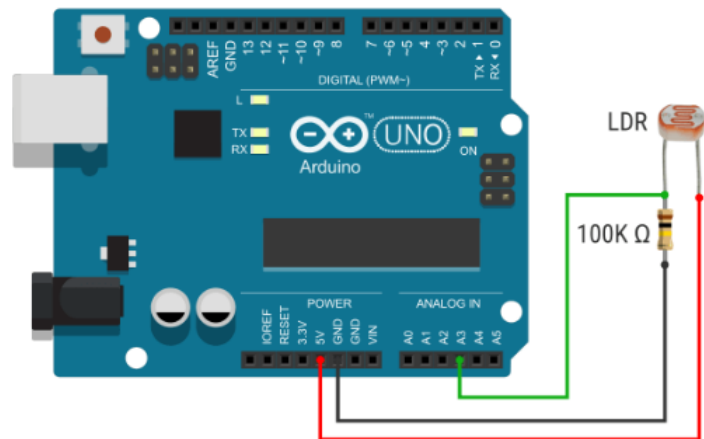
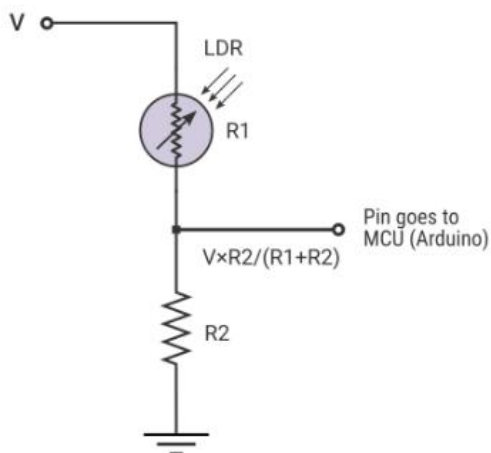
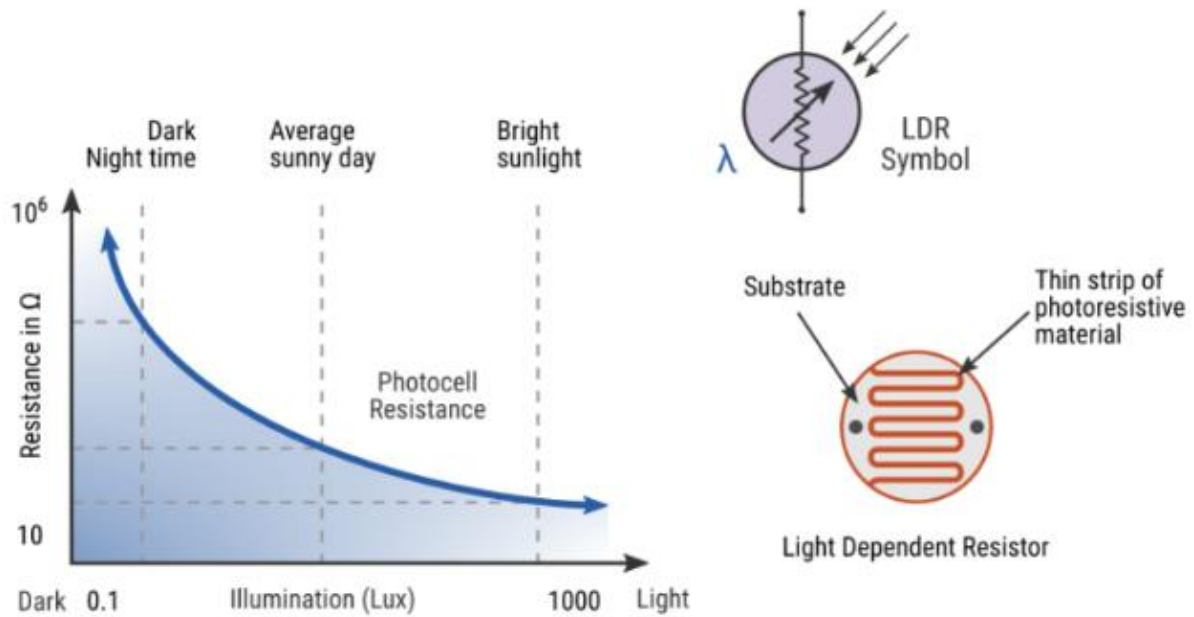
- 7) To interface a joystick with an ESP32 and control the brightness of four LEDs using PWM (Pulse Width Modulation). The brightness of each LED should increase as the joystick moves farther away from the center in a particular direction.



```
1  const int X_PIN = 27;
2  const int Y_PIN = 26;
3
4  const int RED_LED = 4;
5  const int GREEN_LED = 23;
6  const int BLUE_LED = 22;
7  const int YELLOW_LED = 2;
8
9  // Function to turn OFF all LEDs
10 void setup()
11 {
12     Serial.begin(115200);
13
14     pinMode(RED_LED, OUTPUT);
15     pinMode(GREEN_LED, OUTPUT);
16     pinMode(BLUE_LED, OUTPUT);
17     pinMode(YELLOW_LED, OUTPUT);
18
19     Serial.println("Joystick PWM Brightness Control");
20 }
21
22 void loop()
23 {
24     int x = analogRead(X_PIN);
25     int y = analogRead(Y_PIN);
26
27     // Turn OFF all LEDs initially
28     analogWrite(RED_LED,0);
29     analogWrite(GREEN_LED,0);
30     analogWrite(BLUE_LED,0);
31     analogWrite(YELLOW_LED,0);
32
33     Serial.print("X=");
34     Serial.print(x);
35
36     Serial.print(" Y=");
37     Serial.print(y);
38
39     // LEFT
40     if(x < 1800)
41     {
42         int brightness = map(x,1800,0,0,255);
43         brightness = constrain(brightness,0,255);
44
45         analogWrite(RED_LED,brightness);
46
47         Serial.print(" Direction : LEFT");
48         Serial.print(" Brightness : ");
49         Serial.println(brightness);
50     }
51
52     // RIGHT
53     else if(x > 2300)
54     {
55         int brightness = map(x,2300,4095,0,255);
56         brightness = constrain(brightness,0,255);
57
58         analogWrite(GREEN_LED,brightness);
59
60         Serial.print(" Direction : RIGHT");
61         Serial.print(" Brightness : ");
62         Serial.println(brightness);
63     }
64
65     // UP
66     else if(y > 2300)
67     {
68         int brightness = map(y,2300,4095,0,255);
69         brightness = constrain(brightness,0,255);
70
71         analogWrite(BLUE_LED,brightness);
72
73         Serial.print(" Direction : UP");
74         Serial.print(" Brightness : ");
75         Serial.println(brightness);
76     }
77
78     // DOWN
79     else if(y < 1800)
80     {
81         int brightness = map(y,1800,0,0,255);
82         brightness = constrain(brightness,0,255);
83
84         analogWrite(YELLOW_LED,brightness);
85
86         Serial.print(" Direction : DOWN");
87         Serial.print(" Brightness : ");
88         Serial.println(brightness);
89     }
90
91     else
92     {
93         Serial.println(" Direction : CENTER Brightness : 0");
94     }
95
96     delay(50);
97 }
```

AI Based Smart System Development (Real Time Hardware Implementation)
Week – 2 Lecture

- 8) To interface an LDR (Light Dependent Resistor) with an ESP32 and display the light intensity value on the Serial Monitor.



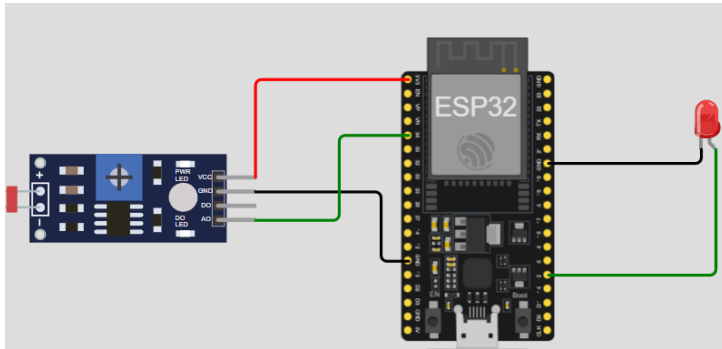
```
1  const int LDR_PIN = 34;
2
3  void setup()
4  {
5      Serial.begin(115200);
6
7      Serial.println("ESP32 LDR Sensor Module");
8  }
9
10 void loop()
11 {
12     int ldrValue = analogRead(LDR_PIN);
13
14     Serial.print("Light Intensity = ");
15     Serial.println(ldrValue);
16
17     delay(500);
18 }
```

AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

- **A₀** for measuring the light intensity (analog value), or
- **D₀** for simple light/dark detection using the onboard comparator and sensitivity adjustment potentiometer.

9) To control the brightness of an LED using the analog output (AO) of an LDR Sensor Module and the PWM feature of the ESP32.



```
1  const int LDR_PIN = 34;
2  const int LED_PIN = 2;
3
4  int ldrValue;
5  int brightness;
6
7  void setup()
8  {
9      Serial.begin(115200);
10
11      pinMode(LED_PIN, OUTPUT);
12
13      Serial.println("-----");
14      Serial.println("ESP32 Automatic Night Lamp");
15      Serial.println("Using LDR Sensor Module");
16      Serial.println("-----");
17  }
18
19  void loop()
20  {
21      // Read Analog Value from LDR Module
22      ldrValue = analogRead(LDR_PIN);
23
24      // Convert ADC (0-4095) to PWM (255-0)
25      // Darkness -> Bright LED
26      brightness = map(ldrValue, 0, 4095, 255, 0);
27
28      // Limit the PWM value
29      brightness = constrain(brightness, 0, 255);
30
31      // Set LED Brightness
32      analogWrite(LED_PIN, brightness);
33
34      // Display values
35      Serial.print("LDR Value : ");
36      Serial.print(ldrValue);
37
38      Serial.print("    LED Brightness : ");
39      Serial.println(brightness);
40
41      delay(200);
42  }
```

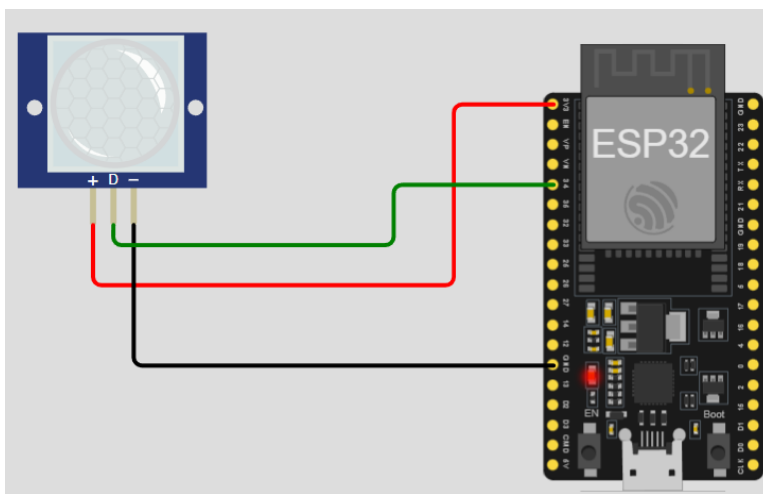
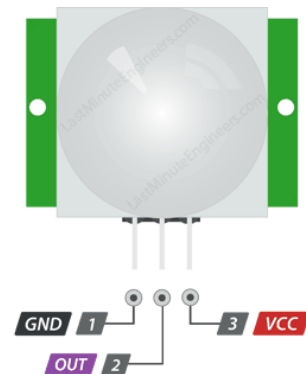
AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

10) To interface a PIR (Passive Infrared) Motion Sensor with an ESP32 and display the motion detection status on the Serial Monitor.

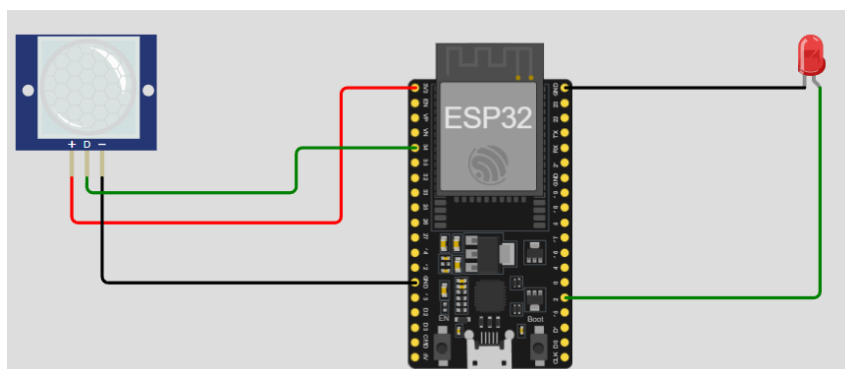
The PIR sensor detects infrared radiation emitted by humans or animals.

- **No Motion:** Output = LOW (0)
- **Motion Detected:** Output = HIGH (1)



```
1  const int PIR_PIN = 34;
2
3  void setup()
4  {
5      Serial.begin(115200);
6
7      pinMode(PIR_PIN, INPUT);
8
9      Serial.println("ESP32 PIR Motion Sensor");
10 }
11
12 void loop()
13 {
14     int motion = digitalRead(PIR_PIN);
15
16     if(motion == HIGH)
17     {
18         Serial.println("Motion Detected");
19     }
20     else
21     {
22         Serial.println("No Motion");
23     }
24
25     delay(500);
26 }
```

11) To interface a PIR motion sensor with an ESP32 and control an LED based on motion detection.



AI Based Smart System Development (Real Time Hardware Implementation)
Week – 2 Lecture

```
1  const int PIR_PIN = 34;
2  const int LED_PIN = 2;
3
4  void setup()
5  {
6      Serial.begin(115200);
7
8      pinMode(PIR_PIN, INPUT);
9      pinMode(LED_PIN, OUTPUT);
10
11     Serial.println("PIR Motion Controlled LED");
12 }
13
14 void loop()
15 {
16     int motion = digitalRead(PIR_PIN);
17
18     if(motion == HIGH)
19     {
20         digitalWrite(LED_PIN, HIGH);
21
22         Serial.println("Motion Detected - LED ON");
23     }
24     else
25     {
26         digitalWrite(LED_PIN, LOW);
27
28         Serial.println("No Motion - LED OFF");
29     }
30
31     delay(100);
32 }
```

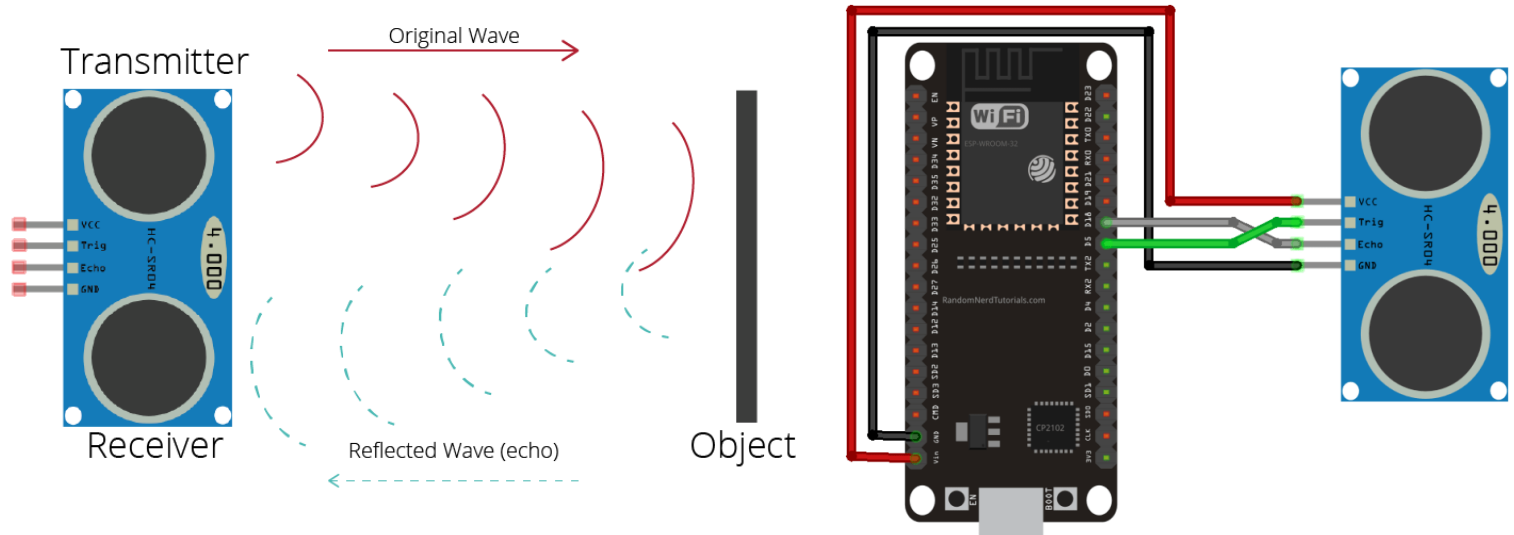
12) To interface an HC-SR04 Ultrasonic Sensor with an ESP32 and measure the distance of an object. The measured distance will be displayed on the Serial Monitor.

Note: The HC-SR04 ECHO pin outputs 5 V. Since the ESP32 GPIO pins are 3.3 V tolerant, use a **voltage divider** (e.g., 1 k Ω and 2 k Ω resistors) or a logic-level shifter to reduce the ECHO signal to about 3.3 V when using real hardware. In Wokwi, this protection is not required.

HC-SR04 Pin	ESP32 Pin
VCC	5V
GND	GND
TRIG	GPIO5
ECHO	GPIO18

AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture



```
1  const int TRIG_PIN = 5;
2  const int ECHO_PIN = 18;
3
4  void setup()
5  {
6      Serial.begin(115200);
7
8      pinMode(TRIG_PIN, OUTPUT);
9      pinMode(ECHO_PIN, INPUT);
10
11     Serial.println("HC-SR04 Distance Measurement");
12 }
13
14 void loop()
15 {
16     // Generate 10 µs trigger pulse
17     digitalWrite(TRIG_PIN, LOW);
18     delayMicroseconds(2);
19
20     digitalWrite(TRIG_PIN, HIGH);
21     delayMicroseconds(10);
22
23     digitalWrite(TRIG_PIN, LOW);
24
25     // Measure echo time
26     long duration = pulseIn(ECHO_PIN, HIGH);
27
28     // Calculate distance
29     float distance = duration * 0.0343 / 2.0;
30
31     Serial.print("Distance = ");
32     Serial.print(distance);
33     Serial.println(" cm");
34
35     delay(500);
36 }
```

AI Based Smart System Development (Real Time Hardware Implementation)
Week – 2 Lecture

13) To develop a Smart Parking Assistant using an HC-SR04 ultrasonic sensor, an RGB LED, and an ESP32. The ultrasonic sensor continuously measures the distance to an obstacle, while the RGB LED indicates how close the vehicle is.

The system measures the distance to the nearest object and classifies it into three zones.

Distance	Status	RGB LED
Greater than 30 cm	Safe	Green
10–30 cm	Caution	Yellow
Less than 10 cm	Stop	Red

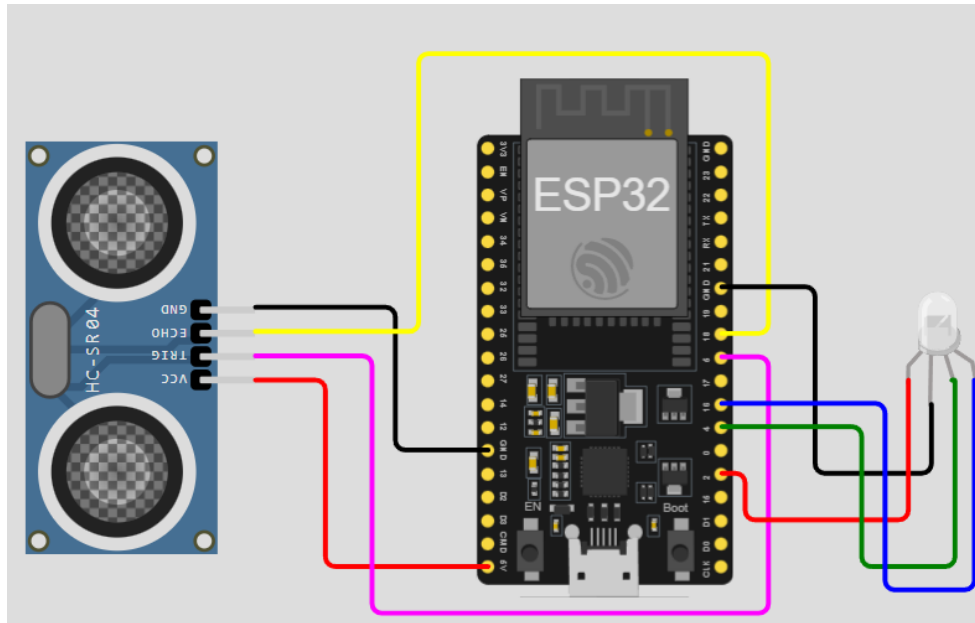
RGB Pin	ESP32
Red	GPIO25
Green	GPIO26
Blue	GPIO27
Common Cathode	GND

```
1  const int TRIG_PIN = 5;
2  const int ECHO_PIN = 18;
3
4  const int RED_PIN = 25;
5  const int GREEN_PIN = 26;
6  const int BLUE_PIN = 27;
7
8  void setColor(int r, int g, int b)
9  {
10     analogWrite(RED_PIN, r);
11     analogWrite(GREEN_PIN, g);
12     analogWrite(BLUE_PIN, b);
13 }
14
15 void setup()
16 {
17     Serial.begin(115200);
18
19     pinMode(TRIG_PIN, OUTPUT);
20     pinMode(ECHO_PIN, INPUT);
21
22     pinMode(RED_PIN, OUTPUT);
23     pinMode(GREEN_PIN, OUTPUT);
24     pinMode(BLUE_PIN, OUTPUT);
25
26     Serial.println("Smart Parking Assistant");
27 }
28
```

```
29 void loop()
30 {
31     // Trigger the sensor
32     digitalWrite(TRIG_PIN, LOW);
33     delayMicroseconds(2);
34
35     digitalWrite(TRIG_PIN, HIGH);
36     delayMicroseconds(10);
37
38     digitalWrite(TRIG_PIN, LOW);
39
40     long duration = pulseIn(ECHO_PIN, HIGH);
41
42     float distance = duration * 0.0343 / 2.0;
43
44     Serial.print("Distance = ");
45     Serial.print(distance);
46     Serial.print(" cm\t");
47
48     if(distance > 30)
49     {
50         setColor(0,255,0);
51
52         Serial.println("SAFE");
53     }
54     else if(distance > 10)
55     {
56         setColor(255,255,0);
57
58         Serial.println("CAUTION");
59     }
60     else
61     {
62         setColor(255,0,0);
63
64         Serial.println("STOP");
65     }
66
67     delay(200);
68 }
```

AI Based Smart System Development (Real Time Hardware Implementation)

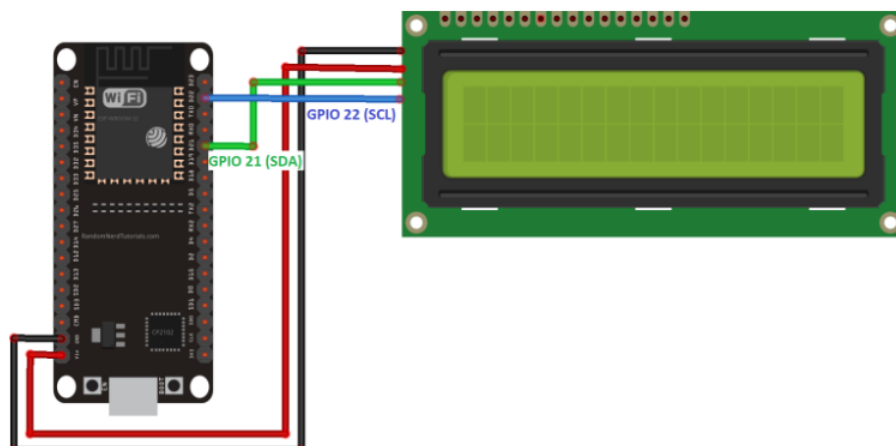
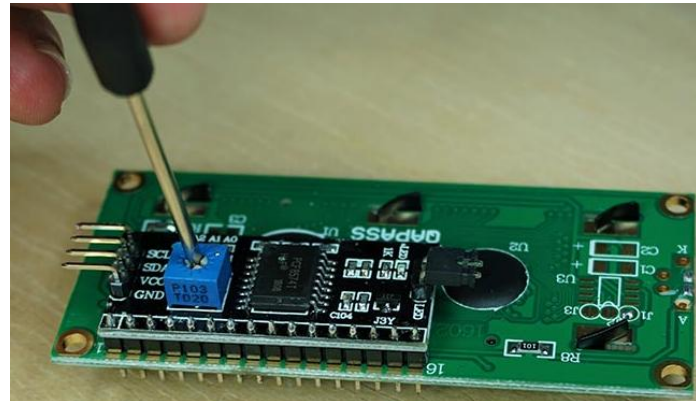
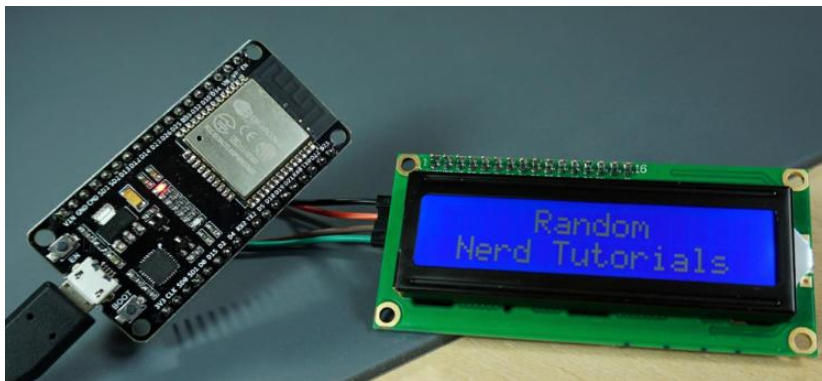
Week – 2 Lecture



14) Interfacing an I2C LCD (16×2) with ESP32

The advantage of using an I2C LCD is that the wiring is really simple. You just need to wire the SDA and SCL pins.

Additionally, it comes with a built-in potentiometer you can use to adjust the contrast between the background and the characters on the LCD. On a “regular” LCD you need to add a potentiometer to the circuit to adjust the contrast.



AI Based Smart System Development (Real Time Hardware Implementation)

Week – 2 Lecture

An **I2C LCD** is a standard 16×2 LCD fitted with an **I2C backpack module** (typically based on the PCF8574 I/O expander). Instead of using 6–10 GPIO pins, the LCD communicates with the ESP32 using only **two data lines**.

The I2C protocol requires only:

- **SDA (Serial Data Line)** – Transfers data.
- **SCL (Serial Clock Line)** – Synchronizes data transmission.

The ESP32 acts as the **I2C Master**, while the LCD module acts as the **I2C Slave**.

Required Library

Install the following library using the Arduino Library Manager:

LiquidCrystal_I2C

```
1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3
4  // LCD I2C Address
5  // Most modules use 0x27 or 0x3F
6
7  LiquidCrystal_I2C lcd(0x27, 16, 2);
8
9  void setup()
10 {
11     Serial.begin(115200);
12
13     lcd.init();           // Initialize LCD
14     lcd.backlight();      // Turn on LCD Backlight
15
16     lcd.setCursor(0,0);
17     lcd.print("Welcome to");
18
19     lcd.setCursor(0,1);
20     lcd.print("ESP32 + I2C LCD");
21
22     Serial.println("LCD Initialized");
23 }
24
25 void loop()
26 {
27
28 }
```

Common I2C Addresses

Module	Address
PCF8574	0x27
PCF8574A	0x3F

If nothing appears on the LCD, scan the I2C bus to find the correct address.